

Data Structure Analysis and Performance Evaluation

Data structures determine how an application stores, retrieves, and changes information. Choosing an appropriate structure can reduce unnecessary traversal and preserve the processing order a task requires. DataStruct Lab demonstrates these decisions through a Python-list-backed stack, a deque-backed queue, and a custom singly linked list. All three require $O(n)$ storage for n values, but their access patterns and memory overhead differ. The complexity estimates assume constant-time equality comparisons and fixed-size references.

A stack uses last in, first out (LIFO): the most recently added value is removed first. This ordering suits undo history, nested processing, and backtracking. `Stack.push` appends to the end of a Python list; `pop` removes that end; `peek` reads it without removal. Push and pop have $O(1)$ amortized cost, although a resizing operation may take $O(n)$, consistent with dynamic-array analysis (Morin, 2013). Searching from the top may still inspect every value.

A queue uses first in, first out (FIFO), making it suitable for printer jobs and messages processed in arrival order. `Queue.enqueue` appends at the rear, while `dequeue` uses `collections.deque.popleft` to remove the front. Endpoint operations take $O(1)$ time without shifting the remaining values (Python Software Foundation, n.d.). Search remains $O(n)$ in the worst case.

A singly linked list stores each value in a Node with data and next fields. Inserting at the head changes only a few references, taking $O(1)$. Deletion by value must first find the matching node; therefore, its worst-case cost is $O(n)$, even though relinking is constant work. Traversal and search are also linear. Linked nodes permit dynamic growth but add reference overhead and lack constant-time indexed access (Morin, 2013).

Implementation and Complexity Prediction

The interface in `app.py` keeps independent structure instances in Streamlit session state. After a user selects an operation, `perform_operation` validates the integer input and calls the appropriate public method. The interface then obtains a shallow snapshot and redraws the diagram. `Stack.to_list` displays top to bottom; `Queue.to_list` displays front to rear; `LinkedList.traverse` follows the node chain. These snapshots avoid exposing private storage or maintaining a second editable copy.

`LinkedList.delete` uses current and previous references. If the first match is the head, it updates the head; otherwise, it connects `previous.next` to `current.next`. A successful deletion decreases the maintained count once and returns `True`. A missing value returns `False`. `Stack` and `Queue` removal or peek raise `IndexError` when empty, and the interface converts that error into friendly feedback. Search leaves all structures unchanged; `size` and `is_empty` are $O(1)$.

The complexity tool in `src/analysis/complexity_analyzer.py` validates the structure, operation, and positive integer size. It selects an explicitly documented rule for one of 20 public operations and returns time bounds, space bounds, an explanation, and three illustrative growth points. It does not infer complexity by timing arbitrary code. For Linked List search at $n = 10,000$, it predicts $O(n)$ worst-case time and relative work of 1, 2, and 4 at n , $2n$, and $4n$.

Storage, auxiliary work, and returned results are reported separately. Linked-list traversal returns an $O(n)$ list while using $O(1)$ traversal bookkeeping; a Stack resize can temporarily allocate $O(n)$ extra storage. These distinctions prevent a constant-time operation from being confused with a constant-size data structure. The implementation guide documents every public method, and Appendix A illustrates three existing tests per structure.

Selection Criteria and Performance Evidence

Selection should begin with operation frequency and access order. A stack fits recent-first processing; a queue preserves arrival order; a linked list fits workloads centered on head insertion and explicit node relationships. Frequent arbitrary lookup or indexed access may justify another structure. Input size, memory overhead, latency requirements, and concurrency needs also matter. The teaching queue is not a complete concurrent message-processing system.

The saved experiment measured Stack push/search, Queue enqueue/search, and Linked List insert/search at 100, 1,000, 10,000, and 50,000 elements. Each case used 30 timed trials and three untimed warmups, producing 24 summaries and 720 samples. Fresh fixtures were created outside the timer; searches used an absent value. The report and charts in Appendices B and C use this project's Windows and CPython 3.14.4 results.

At 50,000 elements, missing-search medians were 243.6 microseconds for Stack, 180.4 for Queue, and 777.4 for Linked List. All three searches are $O(n)$, yet their actual implementations have different costs. From 10,000 to 50,000 elements, those medians grew approximately 5.03, 5.11, and 5.69 times. These observations are consistent with increasing traversal work but do not prove asymptotic bounds or establish a universal speed ranking.

Insertion medians ranged from 100 to 900 nanoseconds across the saved cases, making clock overhead and allocation important. Isolated pushes do not establish amortized cost over a sequence. Big-O predicts growth rather than exact elapsed time, so the report does not assign a prediction-accuracy percentage. Correctness is supported by 128 passing tests; nine representative cases are summarized without listing the entire suite. No single structure is optimal for every application: choose for the workload, then measure its implementation.

References

Morin, P. (2013). *Open data structures: An introduction*. AU Press.

<https://doi.org/10.15215/aupress/9781927356388.01>

Python Software Foundation. (n.d.). *collections - Container datatypes*. Python 3.14

documentation. <https://docs.python.org/3.14/library/collections.html>